In [3]:
```
import os
import json
import glob
import pandas as pd
import numpy as np
```

In [4]:
```
files = glob.glob(
    "/kaggle/input/**/*.json",
    recursive=True
)

print("Total Files:", len(files))

print(files[:5])
```

```
Total Files: 129218
['/kaggle/input/datasets/arpitsaxena9999/synthea-dataset-jsons-
ehr/d8/d8c/d8cddeba-4cf8-412a-86de-b9af0a6a185b.json',
'/kaggle/input/datasets/arpitsaxena9999/synthea-dataset-jsons-
ehr/d8/d8c/d8c4806e-0837-47cc-8500-a1e06e09fc8f.json',
'/kaggle/input/datasets/arpitsaxena9999/synthea-dataset-jsons-
ehr/d8/d8c/d8c1a196-9323-4dbf-a693-ffb68a713189.json',
'/kaggle/input/datasets/arpitsaxena9999/synthea-dataset-jsons-
ehr/d8/d8c/d8c626ec-07f7-4502-bae6-0010033c061a.json',
'/kaggle/input/datasets/arpitsaxena9999/synthea-dataset-jsons-
ehr/d8/d8c/d8cd922c-c607-47e5-801c-ad199061d806.json']
```

In [5]:
```
patients_data = []

observations_data = []

conditions_data = []

encounters_data = []
```

In [6]:

```
for file in files[]:

    try:

        with open(file, "r") as f:

            data = json.load(f)

        entries = data.get("entry", [])

        for entry in entries:

            resource = entry.get("resource", {})

            resource_type = resource.get(
                "resourceType",
                ""
            )

            # ================================================
            # PATIENT RESOURCE
            # ================================================

            if resource_type == "Patient":

                patient_id = resource.get("id")

                gender = resource.get("gender")

                birth_date = resource.get("birthDate")

                city = None
                state = None

                address = resource.get(
                    "address",
                    []
                )

                if len(address) > 0:

                    city = address[0].get("city")

                    state = address[0].get("state")

                patients_data.append({

                    "patient_id": patient_id,

                    "gender": gender,

                    "birth_date": birth_date,
```

```
            "city": city,

            "state": state
        })

    # ================================================
    # OBSERVATION RESOURCE
    # ================================================

    elif resource_type == "Observation":

        subject = resource.get(
            "subject",
            {}
        )

        patient_ref = subject.get(
            "reference",
            ""
        )

        patient_id = (
            patient_ref
            .replace("urn:uuid:", "")
            .replace("Patient/", "")
        )

        observation_name = None

        code = resource.get("code", {})

        coding = code.get(
            "coding",
            []
        )

        if len(coding) > 0:

            observation_name = coding[0].get(
                "display"
            )

        effective_date = resource.get(
            "effectiveDateTime"
        )

        value = None

        if "valueQuantity" in resource:

            value = resource[
                "valueQuantity"
            ].get("value")
```

```
        observations_data.append({

            "patient_id": patient_id,

            "observation_name": observation_name,

            "observation_date": effective_date,

            "value": value
        })

    # ================================================
    # CONDITION RESOURCE
    # ================================================

    elif resource_type == "Condition":

        subject = resource.get(
            "subject",
            {}
        )

        patient_ref = subject.get(
            "reference",
            ""
        )

        patient_id = (
            patient_ref
            .replace("urn:uuid:", "")
            .replace("Patient/", "")
        )

        condition_name = None

        code = resource.get("code", {})

        coding = code.get(
            "coding",
            []
        )

        if len(coding) > 0:

            condition_name = coding[0].get(
                "display"
            )

        onset_date = resource.get(
            "onsetDateTime"
        )
```

```
        conditions_data.append({

            "patient_id": patient_id,

            "condition_name": condition_name,

            "onset_date": onset_date
        })

    # ================================================
    # ENCOUNTER RESOURCE
    # ================================================

    elif resource_type == "Encounter":

        # ----------------------------------
        # PATIENT ID
        # ----------------------------------

        patient_id = None

        subject = resource.get("subject", {})

        if isinstance(subject, dict):

            patient_ref = subject.get(
                "reference",
                ""
            )

            if patient_ref:

                patient_id = (
                    patient_ref
                    .replace("urn:uuid:", "")
                    .replace("Patient/", "")
                )

        # ----------------------------------
        # ENCOUNTER DATE
        # ----------------------------------

        encounter_date = None

        period = resource.get("period", {})

        if isinstance(period, dict):

            encounter_date = period.get("start")

        # ----------------------------------
        # ENCOUNTER TYPE
        # ----------------------------------
```

```
                encounter_type = None


                if not encounter_type:

                    encounter_types = resource.get(
                        "type",
                        []
                    )

                    if len(encounter_types) > 0:

                        first_type = encounter_types[0]

                        if isinstance(first_type, dict):

                            encounter_type = first_type.get(
                                "text"
                            )

                # ----------------------------------
                # APPEND DATA
                # ----------------------------------

                encounters_data.append({

                    "patient_id": patient_id,

                    "encounter_date": encounter_date,

                    "encounter_type": encounter_type
                })

    except Exception as e:

        print(f"Error processing file {file}: {e}")
```

In [7]:

```
patients_df = pd.DataFrame(patients_data)

observations_df = pd.DataFrame(observations_data)

conditions_df = pd.DataFrame(conditions_data)
```

In [8]:
```
patients_df["patient_id"] = (
    patients_df["patient_id"]
    .astype(str)
    .str.replace("urn:uuid:", "", regex=False)
)

observations_df["patient_id"] = (
    observations_df["patient_id"]
    .astype(str)
    .str.replace("urn:uuid:", "", regex=False)
)

conditions_df["patient_id"] = (
    conditions_df["patient_id"]
    .astype(str)
    .str.replace("urn:uuid:", "", regex=False)
)
```

In [9]:
```
patients_df = patients_df.dropna(
    subset=["patient_id"]
)

observations_df = observations_df.dropna(
    subset=["patient_id"]
)

conditions_df = conditions_df.dropna(
    subset=["patient_id"]
)
```

In [10]:
```
patients_df = patients_df.drop_duplicates()

observations_df = observations_df.drop_duplicates()

conditions_df = conditions_df.drop_duplicates()

encounters_df = encounters_df.drop_duplicates()
```

In [11]:
```
patients_df["birth_date"] = pd.to_datetime(
    patients_df["birth_date"],
    errors="coerce",
    utc=True
).dt.tz_localize(None)

observations_df["observation_date"] = pd.to_datetime(
    observations_df["observation_date"],
    errors="coerce",
    utc=True
).dt.tz_localize(None)

conditions_df["onset_date"] = pd.to_datetime(
    conditions_df["onset_date"],
    errors="coerce",
    utc=True
).dt.tz_localize(None)

encounters_df["encounter_date"] = pd.to_datetime(
    encounters_df["encounter_date"],
    errors="coerce",
    utc=True
).dt.tz_localize(None)
```

In [12]:
```
today = pd.Timestamp.today()

patients_df["age"] = (
    (today - patients_df["birth_date"]).dt.days / 365
)

patients_df["age"] = pd.to_numeric(
    patients_df["age"],
    errors="coerce"
)

patients_df["age"] = (
    patients_df["age"]
    .round()
    .astype("Int64")
)
```

In [13]:
```
def age_group(age):

    if pd.isnull(age):
        return "Unknown"

    elif age < 18:
        return "0-17"

    elif age < 35:
        return "18-34"

    elif age < 50:
        return "35-49"

    elif age < 65:
        return "50-64"

    else:
        return "65+"

patients_df["age_group"] = (
    patients_df["age"].apply(age_group)
)
```

In [14]:
```
important_observations = [
    "Body Height",
    "Body Weight",
    "Blood Pressure",
    "Body Mass Index",
    "Glucose",
    "Total Cholesterol"
]

observations_df = observations_df[
    observations_df["observation_name"].isin(
        important_observations
    )
]
```

In [15]:
```
observations_df["year"] = (
    observations_df["observation_date"].dt.year
)

observations_df["month"] = (
    observations_df["observation_date"].dt.month_name()
)

observations_df["month_num"] = (
    observations_df["observation_date"].dt.month
)
```

In [16]:
```
patients_df.head()
```

Out[16]:

	patient_id	gender	birth_date	city	state	age	age_gr
0	f7b73132-64f0-462b-8ca5-94dc5205b297	male	1987-10-12	Monson	MA	39	35-49
1	546cb44f-a105-403e-99ed-bbb3741b82d1	male	1994-02-17	Quincy	MA	32	18-34
2	2df3f23b-6e1c-4ef5-bcd5-bff5e09c14b0	male	2003-12-09	Boston	MA	22	18-34
3	9d8801c1-3b52-4617-99d0-d0dfe6f6af6c	female	1965-11-12	Methuen	MA	61	50-64
4	5721614d-55de-4b09-915a-f53094f75345	female	1965-08-18	Cohasset	MA	61	50-64

In [17]:
```
observations_df.head()
```

Out[17]:

	patient_id	observation_name	observation_date	value	ye
1	f7b73132-64f0-462b-8ca5-94dc5205b297	Body Height	2011-10-29 19:35:57	163.440620	20
2	f7b73132-64f0-462b-8ca5-94dc5205b297	Body Weight	2011-10-29 19:35:57	80.173659	20
3	f7b73132-64f0-462b-8ca5-94dc5205b297	Body Mass Index	2011-10-29 19:35:57	30.013159	20
4	f7b73132-64f0-462b-8ca5-94dc5205b297	Blood Pressure	2011-10-29 19:35:57	NaN	20
6	f7b73132-64f0-462b-8ca5-94dc5205b297	Glucose	2011-10-29 19:35:57	84.000000	20

In [18]:
```
conditions_df.head()
```

Out[18]:

	patient_id	condition_name	onset_date
0	f7b73132-64f0-462b-8ca5-94dc5205b297	Acute bronchitis (disorder)	2000-08-22 02:51:57
1	f7b73132-64f0-462b-8ca5-94dc5205b297	Hypertension	2006-09-20 20:28:19
2	f7b73132-64f0-462b-8ca5-94dc5205b297	Prediabetes	2008-07-28 21:00:46
3	546cb44f-a105-403e-99ed-bbb3741b82d1	Atopic dermatitis	1994-12-01 09:19:27
4	546cb44f-a105-403e-99ed-bbb3741b82d1	Childhood asthma	1997-03-10 02:59:40

In [19]:
```
encounters_df.head()
```

Out[19]:

	patient_id	encounter_date	encounter_type
0	None	2011-10-29 19:35:57	outpatient
1	None	2014-03-06 10:41:14	outpatient
2	None	2017-02-12 18:59:38	outpatient
3	None	2010-03-13 18:32:50	ambulatory
4	None	2010-04-04 19:59:35	ambulatory

In [20]:
```
patients_df.to_csv(
    "/kaggle/working/patients.csv",
    index=False
)

observations_df.to_csv(
    "/kaggle/working/observations.csv",
    index=False
)

conditions_df.to_csv(
    "/kaggle/working/conditions.csv",
    index=False
)

encounters_df.to_csv(
    "/kaggle/working/encounters.csv",
    index=False
)
```

In [21]:
```
observations_df.shape
```

Out[21]: (2544921, 7)